MATH 6610: Advanced Numerical Analysis

Homework 1 — Solutions

Due: Friday, September 25, 2026

Instructions. Show all work. For the computing problem, submit your code and its output together with a short discussion. You may use MATLAB or Python.

Problem 1. Let

$$A = \begin{bmatrix} 1 & 2 & 2 \\ 4 & 4 & 2 \\ 4 & 6 & 4 \end{bmatrix}, \qquad b = \begin{bmatrix} 3 \\ 6 \\ 10 \end{bmatrix}.$$

Compute the factorization $A = LU$ by Gaussian elimination without pivoting, recording the multipliers. Then solve $Ax = b$ by one forward substitution ($Ly = b$) and one back substitution ($Ux = y$). Verify your solution by computing Ax.

Solution. Eliminating column 1 with multipliers $l_{21} = 4$, $l_{31} = 4$ gives rows $(0, -4, -6)$ and $(0, -2, -4)$; eliminating column 2 with $l_{32} = (-2)/(-4) = \frac{1}{2}$ gives $(0, 0, -1)$. Hence

$$L = \begin{bmatrix} 1 & 0 & 0 \\ 4 & 1 & 0 \\ 4 & \frac{1}{2} & 1 \end{bmatrix}, \qquad U = \begin{bmatrix} 1 & 2 & 2 \\ 0 & -4 & -6 \\ 0 & 0 & -1 \end{bmatrix}.$$

Forward substitution: $y_1 = 3$, $y_2 = 6 - 4 \cdot 3 = -6$, $y_3 = 10 - 4 \cdot 3 - \frac{1}{2}(-6) = 1$. Back substitution: $x_3 = 1/(-1) = -1$, $x_2 = (-6 + 6x_3 \cdot (-1))$: from $-4x_2 - 6x_3 = -6$ we get $-4x_2 + 6 = -6$, so $x_2 = 3$; finally $x_1 = 3 - 2 \cdot 3 - 2 \cdot (-1) = -1$. Thus $x = (-1, 3, -1)^T$. Check: $Ax = (-1 + 6 - 2,\ -4 + 12 - 2,\ -4 + 18 - 4)^T = (3, 6, 10)^T$. ∎

Problem 2. Write the elimination matrices as $M_k = I - c^{(k)} e_k^T$, where the vector $c^{(k)}$ contains the multipliers of step k and satisfies $(c^{(k)})_j = 0$ for $j \le k$. Gaussian elimination gives $M_{n-1} \cdots M_2 M_1 A = U$, hence $A = LU$ with $L = M_1^{-1} M_2^{-1} \cdots M_{n-1}^{-1}$. Prove that

$$L = I + \sum_{k=1}^{n-1} c^{(k)} e_k^T,$$

that is, the multipliers drop into place with no interaction between the factors. (Hint: first show $e_j^T c^{(k)} = 0$ for $j \le k$, then expand the product by induction on the number of factors.)

Solution. Since $(c^{(k)})_j = 0$ for $j \le k$, we have $e_j^T c^{(k)} = (c^{(k)})_j = 0$ whenever $j \le k$. Each factor is $M_k^{-1} = I + c^{(k)} e_k^T$ (verified by multiplying out and using $e_k^T c^{(k)} = 0$). We prove by induction on m that

$$P_m := M_1^{-1} \cdots M_m^{-1} = I + \sum_{k=1}^{m} c^{(k)} e_k^T.$$

The case $m = 1$ is the formula for M_1^{-1}. For the step,

$$P_{m+1} = P_m \big(I + c^{(m+1)} e_{m+1}^T\big) = P_m + c^{(m+1)} e_{m+1}^T + \sum_{k=1}^{m} c^{(k)} \underbrace{\big(e_k^T c^{(m+1)}\big)}_{=0} e_{m+1}^T,$$

because $k \le m < m+1$ makes every bracket vanish. Hence $P_{m+1} = I + \sum_{k=1}^{m+1} c^{(k)} e_k^T$, completing the induction. With $m = n-1$ this is the claim: column k of L below the diagonal is exactly $c^{(k)}$, and no fill-in occurs. ■

Problem 3. (Forsythe's example.) Let

$$A = \begin{bmatrix} 10^{-4} & 1 \\ 1 & 1 \end{bmatrix}, \qquad b = \begin{bmatrix} 1 \\ 2 \end{bmatrix}.$$

(a) Solve $Ax = b$ exactly (keep fractions or many digits).

(b) Solve it by LU factorization *without* pivoting, carrying out every arithmetic operation in 3-significant-digit decimal rounding.

(c) Repeat (b) *with* partial pivoting.

(d) Compare the three answers and explain, in terms of the sizes of the intermediate quantities, why (b) fails and (c) succeeds.

Solution. (a) Subtracting the equations, $(1 - 10^{-4})x_1 = 1$, so $x_1 = 1/(1 - 10^{-4}) = 1.00010001\ldots$ and $x_2 = 2 - x_1 = 0.99989998\ldots$

(b) Without pivoting: $l_{21} = 1/10^{-4} = 10^4$ and $u_{22} = 1 - 10^4 = -9999$, which rounds to -1.00×10^4. Forward substitution: $y_1 = 1$, $y_2 = 2 - 10^4 \cdot 1 = -9998$, rounded to -1.00×10^4. Back substitution: $x_2 = (-1.00 \times 10^4)/(-1.00 \times 10^4) = 1.00$, then $x_1 = (1 - 1.00)/10^{-4} = 0$. The computed $x_1 = 0$ has 100% error.

(c) With partial pivoting the rows are exchanged: $l_{21} = 10^{-4}$, $u_{22} = 1 - 10^{-4} \to 1.00$; $y_1 = 2$, $y_2 = 1 - 10^{-4} \cdot 2 = 0.9998 \to 1.00$; $x_2 = 1.00$, $x_1 = 2 - 1.00 = 1.00$. Both components are correct to three digits.

(d) Without pivoting the algorithm manufactures the huge multiplier 10^4; the rounding of u_{22} and y_2 then annihilates the information carried by the second equation (the "1" and the "2" are absorbed). The matrix itself is well conditioned ($\kappa_\infty \approx 4$), so the failure is the algorithm's, not the problem's. Pivoting keeps all multipliers bounded by 1, all intermediate quantities stay of moderate size, and three digits suffice. ■

Problem 4. Suppose Gaussian elimination without pivoting runs to completion on $A \in \mathbb{R}^{n\times n}$ with all pivots $u_{11}, \ldots, u_{nn}$ nonzero, so $A = LU$ with L unit lower triangular. Let A_k denote the leading principal $k \times k$ submatrix of A. Prove that

$$\det A_k = u_{11} u_{22} \cdots u_{kk}, \qquad k = 1, \ldots, n,$$

and conclude that all leading principal minors of A are nonzero. (Hint: partition L and U into blocks and show $A_k = L_k U_k$.)

Solution. Partition

$$L = \begin{bmatrix} L_k & 0 \\ * & * \end{bmatrix}, \qquad U = \begin{bmatrix} U_k & * \\ 0 & * \end{bmatrix},$$

where L_k, U_k are the leading $k \times k$ blocks; the zero blocks are forced by triangularity. Multiplying,

$$LU = \begin{bmatrix} L_k U_k & * \\ * & * \end{bmatrix},$$

so the leading $k \times k$ block of $A = LU$ is $A_k = L_k U_k$. Taking determinants, $\det A_k = \det L_k \det U_k = 1 \cdot u_{11} \cdots u_{kk}$, since L_k is unit lower triangular and U_k is upper triangular with the first k pivots on its diagonal. As all pivots are nonzero, every $\det A_k \neq 0$. ■

Problem 5. (Computing.) Implement LU factorization with partial pivoting from scratch (do not call `lu`), returning P, L, U.

(a) Test your code on a random 200×200 matrix with standard normal entries: report $\max_{ij} |(PA - LU)_{ij}|$ and $\max_{jk} |l_{jk}|$.

(b) Report the growth factor $\rho = \max_{jk} |u_{jk}| \,/\, \max_{jk} |a_{jk}|$ for ten random matrices. How do your values compare with the worst-case bound 2^{n-1}?

(c) Solve $Ax = b$ with your factors for a b constructed so that the exact solution is $x = (1, \ldots, 1)^T$, and report $\left\| x_{\text{computed}} - x \right\|_\infty$.

Solution. A correct implementation loops over columns $k = 1, \ldots, n-1$, selects the pivot row $\operatorname{argmax}_{i \geq k} |u_{ik}|$, swaps rows of U, of the computed part of L, and of P, then eliminates with multipliers $l_{jk} = u_{jk}/u_{kk} \leq 1$ in modulus. Typical results in double precision for $n = 200$: $\max |(PA - LU)_{ij}| \approx 10^{-14}$ (backward error at the level of $n\, \varepsilon_{\text{mach}}\, \|A\|$); $\max |l_{jk}| = 1$ up to rounding, confirming the pivoting bound. (b) Growth factors for Gaussian random matrices typically fall between about 5 and 20 for $n = 200$, many orders of magnitude below $2^{199} \approx 8 \times 10^{59}$: the worst case is not what random matrices do. (c) With $b = A\mathbf{1}$, the computed solution satisfies $\|x_c - \mathbf{1}\|_\infty \approx \kappa_\infty(A)\, \varepsilon_{\text{mach}}$, typically 10^{-13} to 10^{-12} for such matrices, consistent with the accuracy theorem. ■